Projeto Interdisciplinar

Memorial Descritivo do Modelo NoAM

Centro Estadual de Educação Tecnológica "Paula Souza"

Instituição: Faculdade de Tecnologia (FATEC)

Cidade: Franca-SP

Curso: Desenvolvimento de Software Multiplataforma

Grupo: 07

Alunos:

- Gabriel de Sousa Khodor (GitHub: Gabrielkhodor)
- Jose Paulo Archetti Conrado (GitHub: ppconrado)

Disciplina: Banco de Dados Não Relacional

Turma: DSM P3 G07 2026-1

Professor: Mestre Fausto Cintra

Data: Abril de 2026

Sumário

1. Introdução
2. Seção A. Memorial Descritivo do Projeto
3. Seção B. Descrição das Entidades e Campos
4. Seção C. Justificativa das Relações e Cardinalidades
5. Considerações Finais
6. Referências

---

1. Introdução

```
Este memorial descritivo apresenta a documentação conceitual do Projeto
Interdisciplinar, fundamentada no modelo NoAM e sincronizada com a versão vigente
do diagrama UML oficial do sistema. O documento consolida a visão de domínio do
sistema de gerenciamento de palestras e cursos, detalhando entidades, atributos,
relacionamentos, cardinalidades e decisões de modelagem.

O objetivo deste material consiste em registrar, de forma técnica e justificável,
a estrutura de dados e as regras de negócio que sustentam a implementação do
backend.
```

2. Seção A - Memorial Descritivo do Projeto Interdisciplinar

2.1 Tema: Sistema de Gerenciamento de Palestras e Cursos

O tema do projeto é o desenvolvimento de um sistema de gerenciamento de eventos acadêmicos, contemplando os principais processos de:

- criação e manutenção de eventos;
- cadastro de usuários (administradores e participantes) e palestrantes;
- organização de sessões/aulas por evento;
- inscrição de participantes em eventos;
- controle de presença por sessão;
- emissão e validação de certificados digitais.

2.2 Justificativa

A gestão de eventos acadêmicos envolve atividades repetitivas e sensíveis a erro quando executadas de forma manual, como controle de inscritos, associação de palestrantes, acompanhamento de presença e emissão de certificados. Esse cenário pode produzir inconsistências, retrabalho e baixa rastreabilidade.

Propõe-se, portanto, centralizar esses fluxos em uma estrutura única e padronizada, de modo a permitir:

- maior confiabilidade dos dados;
- melhor organização do processo administrativo;
- suporte à auditoria e histórico de participação;
- escalabilidade para novas regras de negócio, como diferentes critérios de certificação.

2.3 Objetivos

2.3.1 Objetivo Geral

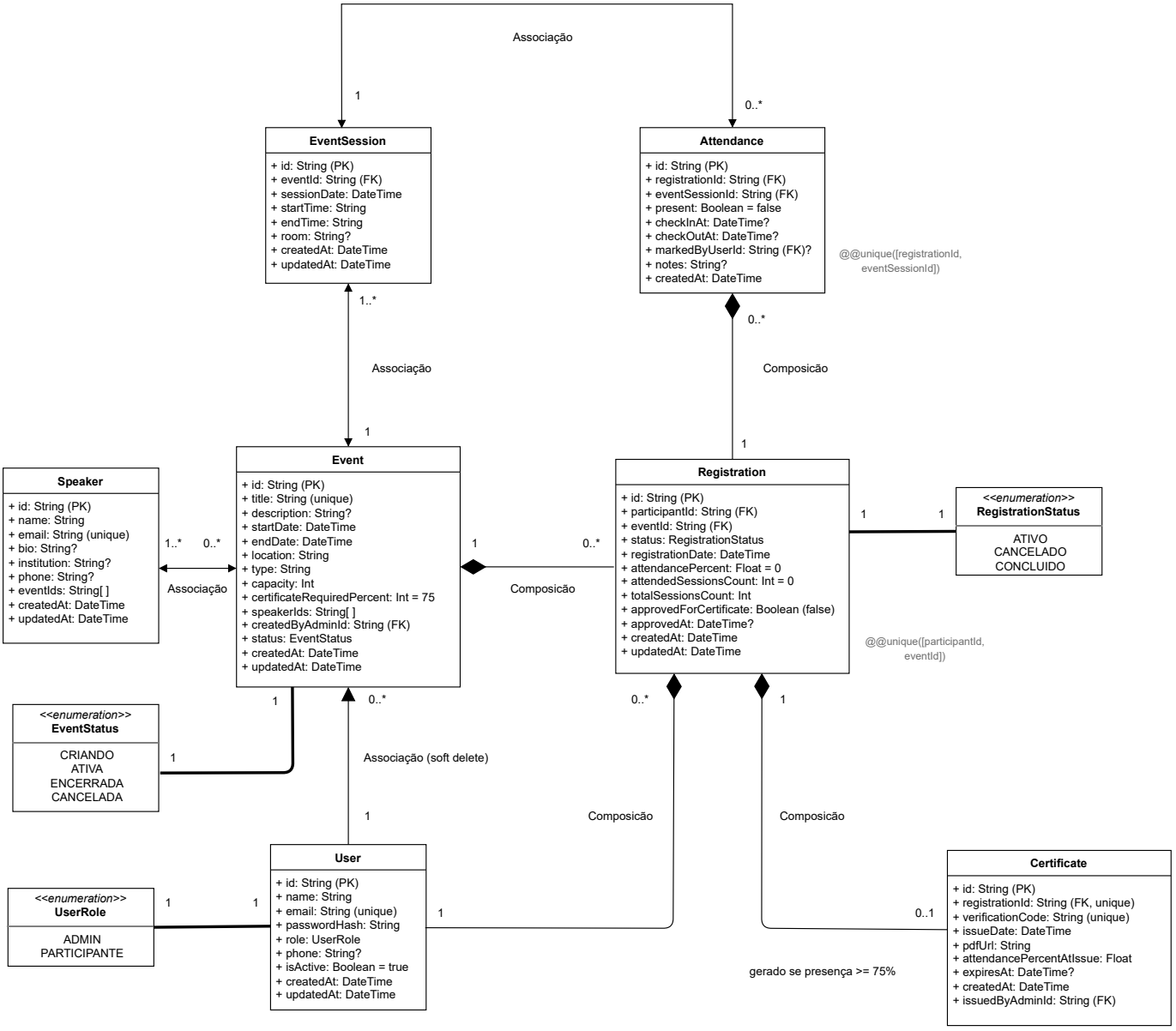
Desenvolver uma base de dados e regras de relacionamento capazes de sustentar um sistema de gerenciamento de palestras e cursos com integridade, rastreabilidade e consistência semântica.

2.3.2 Objetivos Específicos

- modelar entidades nucleares do domínio de eventos;
- representar a associação entre palestrantes e eventos;
- controlar o ciclo de inscrições por status padronizado;
- controlar presença de forma granular por sessão;
- permitir emissão de certificados vinculada a regras de presença;
- manter consistência entre o modelo conceitual, o diagrama UML e a implementação da API.

3. Seção B - Descrição de Cada Entidade e de Cada Campo

Diagrama de referência utilizado nesta seção:



3.1 Entidade User

Finalidade: representar os usuários do sistema, classificados pelos perfis ADMIN ou PARTICIPANTE.

Campos:

- id (String, obrigatório): identificador único do usuário.
- name (String, obrigatório): nome completo.
- email (String, obrigatório e único): e-mail principal.
- passwordHash (String, obrigatório): hash da senha de autenticação.
- role (UserRole, obrigatório): perfil de acesso do usuário.
- phone? (String, opcional): telefone para contato.
- isActive (Boolean, obrigatório, padrão true): controle de ativação.
- createdAt (DateTime, obrigatório): data e hora de criação.
- updatedAt (DateTime, obrigatório): data e hora da última atualização.

3.2 Entidade Speaker

Finalidade: representar palestrantes vinculados aos eventos.

Campos:

- id (String, obrigatório): identificador único do palestrante.
- name (String, obrigatório): nome completo.
- email (String, obrigatório e único): e-mail de contato.
- bio? (String, opcional): biografia resumida.
- institution? (String, opcional): instituição de origem.
- phone? (String, opcional): telefone.
- eventIds (String[ ], obrigatório): lista de IDs dos eventos em que atua (relação N:N com Event via array MongoDB).
- createdAt (DateTime, obrigatório): data de criação.
- updatedAt (DateTime, obrigatório): data da última atualização.

3.3 Entidade Event

Finalidade: armazenar dados dos eventos do tipo palestra, curso ou workshop.

Campos:

- id (String, obrigatório): identificador único do evento.
- title (String, obrigatório e único): título do evento.
- description? (String, opcional): descrição do evento.
- startDate (DateTime, obrigatório): data inicial.
- endDate (DateTime, obrigatório): data final.
- location (String, obrigatório): local físico ou virtual.
- type (String, obrigatório): classificação do evento (palestra, workshop, curso etc.).
- capacity (Int, obrigatório): capacidade máxima de participantes.
- certificateRequiredPercent (Int, obrigatório, padrão 75): percentual mínimo de presença para certificação.
- speakerIds (String[ ], obrigatório): lista de IDs dos palestrantes que atuam no evento (relação N:N com Speaker via array MongoDB).
- createdByAdminId (String, obrigatório, FK): referência ao usuário administrador criador.
- status (EventStatus, obrigatório): estado do evento.
- createdAt (DateTime, obrigatório): data de criação.
- updatedAt (DateTime, obrigatório): data da última atualização.

3.4 Entidade EventSession

Finalidade: representar as sessões (aulas) de um evento, permitindo controle de presença granular.

Justificativa de criação da entidade:

- a entidade Event, isoladamente, não permite rastrear frequência por encontro quando um curso possui várias aulas em datas e horários distintos;
- EventSession viabiliza a representação formal do cronograma real (data, horário e sala por sessão);

- a presença passa a ser registrada em nível de sessão (Attendance), tornando a apuração objetiva e auditável;
- a regra de certificação baseada em percentual mínimo torna-se tecnicamente verificável a partir de evidências granulares.

Campos:

- id (String, obrigatório): identificador único da sessão.
- eventId (String, obrigatório, FK): referência ao evento.
- sessionDate (DateTime, obrigatório): data da sessão.
- startTime (String, obrigatório): horário de início.
- endTime (String, obrigatório): horário de término.
- room? (String, opcional): sala/local específico.
- createdAt (DateTime, obrigatório): data de criação.
- updatedAt (DateTime, obrigatório): data da última atualização.

3.5 Entidade Registration

Finalidade: registrar a inscrição de um participante em um evento.

Campos:

- id (String, obrigatório): identificador único da inscrição.
- participantId (String, obrigatório, FK): referência ao usuário participante.
- eventId (String, obrigatório, FK): referência ao evento.
- status (RegistrationStatus, obrigatório): estado da inscrição.
- registrationDate (DateTime, obrigatório): data e hora da inscrição.
- attendancePercent (Float, obrigatório, padrão 0): percentual de presença acumulado.
- attendedSessionsCount (Int, obrigatório, padrão 0): quantidade de sessões com presença.
- totalSessionsCount (Int, obrigatório): total de sessões consideradas.
- approvedForCertificate (Boolean, obrigatório, padrão false): indica se foi aprovado para emissão de certificado.
- approvedAt? (DateTime, opcional): data de aprovação para certificado.
- createdAt (DateTime, obrigatório): data de criação.
- updatedAt (DateTime, obrigatório): data da última atualização.

Regra adicional de integridade: combinação única por participante e evento (participantId + eventId).

3.6 Entidade Attendance

Finalidade: controlar presença por sessão para cada inscrição.

Campos:

- id (String, obrigatório): identificador único do registro de presença.
- registrationId (String, obrigatório, FK): referência à inscrição.
- eventSessionId (String, obrigatório, FK): referência à sessão.
- present (Boolean, obrigatório, padrão false): indica presença na sessão.
- checkInAt? (DateTime, opcional): horário de check-in.
- checkOutAt? (DateTime, opcional): horário de check-out.

- markedByUserId? (String, opcional, FK): usuário que marcou a presença.
- notes? (String, opcional): observações de registro.
- createdAt (DateTime, obrigatório): data de criação.

Regra adicional de integridade: combinação única por inscrição e sessão (registrationId + eventSessionId).

3.7 Entidade Certificate

Finalidade: armazenar o certificado emitido para uma inscrição aprovada.

Argumentação de modelagem para manutenção da entidade própria:

- Registration representa o processo de inscrição e a apuração de elegibilidade do participante;
- Certificate representa um artefato de saída, emitido somente após validação das regras de frequência;
- os ciclos de vida são distintos (processo x resultado), o que justifica a separação por responsabilidade;
- a cardinalidade 1:0..1 com Registration expressa com clareza que nem toda inscrição gera certificado;
- a separação evita concentrar em Registration campos que só fazem sentido após aprovação (verificationCode, issueDate, pdfUrl e attendancePercentAtIssue);
- o modelo separado favorece evolução futura para reemissão, segunda via, invalidação de código, histórico e auditoria.

Campos:

- id (String, obrigatório): identificador único do certificado.
- registrationId (String, obrigatório, FK e único): referência da inscrição vinculada.
- verificationCode (String, obrigatório e único): código de verificação de autenticidade.
- issueDate (DateTime, obrigatório): data de emissão.
- pdfUrl (String, obrigatório): endereço do arquivo PDF.
- attendancePercentAtIssue (Float, obrigatório): percentual de presença no momento da emissão.
- expiresAt? (DateTime, opcional): data de expiração, quando aplicável.
- createdAt (DateTime, obrigatório): data de criação.
- issuedByAdminId (String, obrigatório, FK): usuário administrador emissor.

3.8 Enumerações

3.8.1 RegistrationStatus

Valores definidos:

- ATIVO
- CANCELADO
- CONCLUIDO

O enum RegistrationStatus padroniza o estado da inscrição do participante no evento, controlando o ciclo de vida da inscrição.

Significado dos valores:

1. ATIVO: inscrição válida e vigente, participante apto a participar das sessões.
2. CANCELADO: inscrição foi cancelada (pelo participante ou pela administração), deixando de valer operacionalmente.

3. CONCLUIDO: inscrição finalizada após o ciclo do evento/participação, normalmente com frequência já apurada.

Propósito prático no sistema:

1. Governar regras: definir quando pode registrar presença, cancelar, concluir, etc.
2. Evitar inconsistência: impede status em texto livre com grafias diferentes.
3. Apoiar certificados: ajuda a separar quem ainda está em andamento de quem finalizou.
4. Facilitar relatórios: métricas claras de inscritos ativos, cancelados e concluídos.

3.8.2 EventStatus

Valores definidos:

- CRIANDO
- ATIVA
- ENCERRADA
- CANCELADA

O propósito do enum EventStatus é padronizar o ciclo de vida do evento e evitar estados ambíguos no sistema.

Cada valor representa uma fase:

1. CRIANDO: evento em cadastro/configuração, ainda não disponível para operação normal.
2. ATIVA: evento publicado e em andamento, permitindo inscrições e controle de presença conforme regras.
3. ENCERRADA: evento finalizado, sem novas interações operacionais (ex.: presença/inscrição), mantendo histórico.
4. CANCELADA: evento interrompido/invalidado, com comportamento diferente de encerramento normal.

Na prática, esse enum serve para:

1. Regras de negócio: bloquear/liberar ações conforme fase.
2. Integridade de dados: impedir textos livres e variações (“ativo”, “ativa”, “em andamento”).
3. Consulta e relatórios: filtrar eventos por status de forma consistente.
4. Manutenção do código: facilitar validações no backend e manter comportamento previsível.

3.8.3 UserRole

Valores definidos:

- ADMIN
- PARTICIPANTE

4. Seção C - Justificativa do Tipo de Relação e Cardinalidades

4.1 Relação Speaker - Event

- Tipo: Associação bidirecional N:N.
- Cardinalidade: Speaker (1..*) para Event (0..*).

Justificativa:

- um palestrante pode atuar em múltiplos eventos;
- um evento pode contar com múltiplos palestrantes;
- a relação permite flexibilidade na composição de grade de palestras por evento.

4.2 Relação User (ADMIN) - Event

- Tipo: Associação (com estratégia de soft delete para usuário administrador).
- Cardinalidade: User (1) para Event (0..*), e cada Event pertence a 1 usuário administrador criador.

Justificativa:

- cada evento requer um responsável administrativo;
- a referência createdByAdminId garante autoria e auditoria.
- a exclusão física do usuário administrador não é adotada como comportamento padrão operacional; prioriza-se desativação (isActive = false) para preservação histórica.
- no schema Prisma, a relação `Event.createdByAdmin -> User.id` vai ser configurada como `onDelete: Restrict`, evitando remoção em cascata dos eventos ao excluir um usuário.

4.3 Relação User (PARTICIPANTE) - Registration

- Tipo: Composição.
- Cardinalidade: User (1) para Registration (0..*), e cada Registration pertence a 1 participante.

Justificativa:

- as inscrições existem vinculadas ao participante;
- um participante pode possuir várias inscrições em eventos distintos.

4.4 Relação Event - Registration

- Tipo: Composição.
- Cardinalidade: Event (1) para Registration (0..*), e cada Registration pertence a 1 Event.

Justificativa:

- a inscrição depende da existência do evento;
- sem evento não existe contexto válido para inscrição.

4.5 Relação Registration - Attendance

- Tipo: Composição.
- Cardinalidade: Registration (1) para Attendance (0..*), e cada Attendance pertence a 1 Registration.

Justificativa:

- os registros de presença dependem da inscrição;
- a granularidade por sessão permite calcular o percentual de frequência com precisão.

4.6 Relação Event - EventSession

- Tipo: Associação estrutural do evento com suas sessões.
- Cardinalidade: Event (1) para EventSession (1..*), e cada EventSession pertence a 1 Event.

Justificativa:

- um evento pode ter uma ou várias sessões;
- o detalhamento por sessão é necessário para o controle efetivo de frequência.

Detalhamento técnico complementar:

- sem EventSession, a frequência ficaria agregada no nível do evento, sem rastreabilidade por aula;
- com EventSession, cada encontro torna-se uma unidade formal de controle para registros de Attendance;
- essa modelagem suporta o cálculo objetivo do percentual de presença:

```
attendancePercent = (attendedSessionsCount / totalSessionsCount) X 100
```

- o percentual apurado é utilizado de forma transparente na decisão de emissão do certificado.

4.7 Relação EventSession - Attendance

- Tipo: Associação.
- Cardinalidade: EventSession (1) para Attendance (0..*), e cada Attendance pertence a 1 EventSession.

Justificativa:

- cada marca de presença ocorre em uma sessão específica;
- uma sessão pode conter várias marcações de presença, uma por inscrição.

4.8 Relação Registration - Certificate

- Tipo: Composição.
- Cardinalidade: Registration (1) para Certificate (0..1), e cada Certificate pertence a 1 Registration.

Justificativa:

- o certificado é derivado da inscrição;
- nem toda inscrição gera certificado, pois a emissão depende de regra de aprovação por presença;
- registrationId único em Certificate garante, no máximo, um certificado por inscrição.

Argumentação complementar da decisão arquitetural:

- incorporar os dados de Certificate dentro de Registration simplifica apenas cenários muito pequenos e com baixa expectativa de evolução;
- essa alternativa, contudo, mistura dados de participação com dados de emissão documental, reduzindo coesão do modelo;
- a separação entre Registration (vínculo e elegibilidade) e Certificate (emissão do documento) preserva clareza semântica;
- a estrutura separada reduz campos nulos em Registration e melhora manutenção do schema no médio e longo prazo;
- para evoluções funcionais, a entidade Certificate própria apresenta melhor escalabilidade e rastreabilidade.

4.9 Relações entre Enumerações e Entidades

- RegistrationStatus - Registration: cardinalidade 1:1 por instância de inscrição (cada inscrição possui exatamente um status).
- EventStatus - Event: cardinalidade 1:1 por instância de evento (cada evento possui exatamente um status).
- UserRole - User: cardinalidade 1:1 por instância de usuário (cada usuário possui exatamente um papel).

Justificativa:

- o uso de enumerações padroniza valores válidos;
- evita inconsistências semânticas e facilita validação de regras de negócio.

Observação de modelagem:

- o controle de certificação considera o campo attendancePercent e a regra mínima definida em certificateRequiredPercent (padrão 75).

5. Considerações Finais

O modelo NoAM/UML final contempla os elementos essenciais para o domínio de gestão de palestras e cursos, com consistência entre entidades, relacionamentos, cardinalidades e regras de negócio. A inclusão de EventSession e Attendance fortalece o controle de frequência e viabiliza a emissão de certificados com base em critério objetivo de presença.

A estrutura proposta encontra-se aderente ao backend orientado a MongoDB/Prisma, mantendo rastreabilidade, integridade e potencial de expansão futura.

6. Referências

- Diagrama UML oficial do sistema: docs/UML-Gerenciamento-Palestras-Estrito.drawio.
- Especificação de persistência de dados: back-end/prisma/schema.prisma.
- Documento de apoio arquitetural: docs/RESUMO-ARQUITETURA-FINAL.md.